

Assignment Questions

Course Code & Name : S.MAHALAKSHMI(192524372)CSA1205 - Computer Architecture

Assignment Title:	Design and Performance Analysis of a Pipeline Processor with Data Hazard Detection, Forwarding, and Memory Hierarchy
Course Outcome(s) — CO:	CO3: Apply register transfer notation, arithmetic/logic operations, and multiple-bus organization to model inter-unit data movement in a processor datapath. Design a pipelined and superscalar execution scheme, analyzing pipeline stages, hazards (data, instruction), out-of-order and speculative execution, and dispatch/execution units. CO4: Evaluate memory hierarchy design choices — cache levels (L1/L2/L3), virtual memory/paging, and advanced memory technologies — using latency, bandwidth, and throughput metrics.
Bloom's Taxonomy Level	L4 – Analyze; L6 – Create (design-based)
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure

Problem Statement :

A processor design team is prototyping a new 5-stage RISC pipeline (IF, ID, EX, MEM, WB) together with its supporting memory hierarchy, and needs to verify, on paper, that the pipeline correctly handles data hazards through forwarding and that the accompanying memory (cache) design does not undermine overall performance. You are given a short instruction sequence containing back-to-back dependent instructions, including at least one load instruction whose result is used by the very next instruction (e.g., LOAD R1, 0(R2) followed by ADD R4, R1, R5), which will stall the naive pipeline. First, using register transfer notation, model how each instruction moves through the five pipeline stages cycle by cycle — including how the MEM stage accesses the memory system — and draw a pipeline timing diagram for the unmodified pipeline, clearly marking every data hazard (RAW dependency) that occurs. Second, design a hazard detection unit that identifies these dependencies by comparing source and destination register fields across pipeline stages, and specify the control signals it generates. Third, design the forwarding (bypass) paths needed to resolve each hazard without stalling — specifically EX/MEM-to-EX and MEM/WB-to-EX forwarding — and show, with a revised timing diagram, how forwarding eliminates or reduces stalls; identify the load-use hazard that forwarding alone cannot resolve and design the minimal-stall solution for it. Fourth, design the cache memory system that services the MEM stage: choose an L1 cache size, mapping technique, and replacement policy suited to this workload, and state the cache-hit and cache-miss access latencies (in cycles) you will assume. Fifth, compute the CPI of your instruction sequence under three scenarios: stall-only hazard handling, full forwarding with all cache accesses hitting, and full forwarding with a given cache miss rate causing additional pipeline stalls on load/store instructions — and calculate the resulting speedup and pipeline efficiency of your combined forwarding-and-cache design over the stall-only approach. Finally, generalize your findings: for a longer representative program with a given percentage of dependent instruction pairs and a given cache hit rate, estimate the overall CPI and throughput your design would achieve, and discuss how improving the cache hit rate versus improving forwarding coverage each affect performance.

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech – Computer Science and Engineering
Course Code & Course Name	CSA1205 – Computer Architecture
Academic Year / Batch	2026–2027 /
Faculty Name	
Assignment Title	Design and Performance Analysis of a Pipeline Processor with Data Hazard Detection, Forwarding, and Memory Hierarchy
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO3 and CO4
Bloom's Taxonomy Level	L4 – Analyze; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processor and memory systems support high-performance digital infrastructure, embedded systems and computing applications.

B. Assignment Problem / Challenge

The assignment presents a realistic computing and processor-design challenge. A processor design team is prototyping a 5-stage RISC pipeline with IF, ID, EX, MEM and WB stages. The design must correctly handle back-to-back dependent instructions using hazard detection and forwarding, while ensuring that the memory hierarchy does not reduce overall pipeline performance.

The assignment requires the students to:

- Apply Computer Architecture concepts rather than reproduce theory.
- Identify and analyze RAW data dependencies in the instruction sequence.
- Consider performance requirements, hardware constraints and memory-access delays.
- Develop and compare stall-only and forwarding-based solutions.
- Use modern computing/design tools to perform calculations, testing and documentation.
- Interpret performance results and justify the final processor and cache design.

C. Problem Statement

Problem / Case / Design Challenge:

A processor design team is prototyping a new 5-stage RISC pipeline (IF, ID, EX, MEM, WB) together with its supporting memory hierarchy. The team must verify that the pipeline correctly handles data hazards through forwarding and that the accompanying cache design does not undermine overall performance.

The given instruction sequence contains back-to-back dependent instructions, including a load instruction whose result is used by the immediately following instruction. The work requires register-transfer modelling, cycle-by-cycle pipeline timing, RAW hazard identification, hazard detection logic, forwarding-path design, minimal-stall handling of the load-use hazard, L1 cache design, CPI calculation, speedup and pipeline-efficiency analysis.

The instruction sequence used for analysis is:

```
I1: LOAD R1, 0(R2)      ; R1 ← M[R2]
I2: ADD  R4, R1, R5      ; R4 ← R1 + R5
I3: SUB  R6, R4, R7      ; R6 ← R4 - R7
I4: AND  R8, R6, R9      ; R8 ← R6 AND R9
```

I5: ADD R10, R4, R6 ; R10 $\leftarrow$ R4 + R6

D. Requirements and Constraints

Functional Requirements

- Model instruction movement through IF, ID, EX, MEM and WB stages.
- Identify all RAW dependencies in the instruction sequence.
- Detect hazards by comparing source and destination register fields.
- Provide EX/MEM-to-EX and MEM/WB-to-EX forwarding paths.
- Insert a minimum stall for the immediate load-use hazard.
- Design an L1 cache to service the MEM stage.
- Calculate CPI, speedup and pipeline efficiency for multiple scenarios.

Performance Requirements

- Ideal pipeline CPI should be 1.
- Forwarding should eliminate ordinary ALU-to-ALU dependency stalls.
- The load-use dependency should require only one bubble.
- L1 cache hits should complete within the assumed 1-cycle MEM stage.
- Cache misses should be included through the specified miss penalty.

Technical Constraints and Assumptions

Parameter	Selected / Assumed Value
Pipeline stages	5
Pipeline organization	IF, ID, EX, MEM, WB
Ideal CPI	1
Load-use dependency	1 stall cycle with forwarding
L1 cache size	32 KB
Mapping technique	4-way set associative
Block size	64 bytes
Replacement policy	LRU
Cache hit latency	1 cycle
Miss penalty	20 additional cycles
Example L1 miss rate	5%

E. Student Work

1. Problem Understanding and Formulation

What is the problem?

The main problem is to improve the performance of a 5-stage RISC pipeline when consecutive instructions depend on results produced by earlier instructions. Without special hardware support, dependent instructions must wait until the producer writes its result back to the register file, causing stalls and increasing CPI.

Expected Outcomes

- Correct identification of RAW hazards.
- A working hazard detection approach.
- Forwarding paths that resolve most ALU dependencies.
- Minimal stalling for the immediate load-use dependency.
- An L1 cache design suitable for the MEM stage.

- Quantitative comparison using CPI, speedup and efficiency.

Available Information / Data

- A five-instruction RISC sequence with multiple dependencies.
- Five pipeline stages: IF, ID, EX, MEM and WB.
- Specified forwarding paths: EX/MEM-to-EX and MEM/WB-to-EX.
- Assumed cache hit latency, miss penalty and miss rate.

Assumptions

The ideal CPI is 1. The pipeline has five stages. A load-use dependency requires one bubble even with forwarding. The L1 cache hit latency is 1 cycle, the miss penalty is 20 additional cycles and the example miss rate is 5%.

Constraints to be Satisfied

- Forwarding cannot completely eliminate the immediate load-use hazard.
- Hardware complexity should remain reasonable.
- The cache must provide fast access while reducing conflict misses.
- The solution must improve performance compared with the stall-only approach.

2. Application of Course Knowledge

Register Transfer Modelling

Pipeline Stage	Register Transfer Operation
IF	$IR \leftarrow IM[PC]; NPC \leftarrow PC + 4$
ID	$A \leftarrow Reg[rs]; B \leftarrow Reg[rt]; Imm \leftarrow SignExtend(immediate)$
EX for LOAD	$ALUOut \leftarrow A + Imm$
MEM for LOAD	$LMD \leftarrow Cache[ALUOut]$
WB for LOAD	$Reg[rt] \leftarrow LMD$
EX for ADD	$ALUOut \leftarrow A + B$
EX for SUB	$ALUOut \leftarrow A - B$
EX for AND	$ALUOut \leftarrow A \text{ AND } B$
MEM for ALU instruction	ALUOut passes through; no data-memory read
WB for ALU instruction	$Reg[rd] \leftarrow ALUOut$

RAW Data Hazard Identification

Dependency	Register	Hazard
I1 LOAD $\rightarrow$ I2 ADD	R1	RAW load-use hazard
I2 ADD $\rightarrow$ I3 SUB	R4	RAW hazard
I3 SUB $\rightarrow$ I4 AND	R6	RAW hazard
I2 ADD $\rightarrow$ I5 ADD	R4	RAW hazard
I3 SUB $\rightarrow$ I5 ADD	R6	RAW hazard

Hazard Detection Concept

The hazard detection unit compares the source register fields of the instruction in IF/ID with the destination register fields of older instructions. The important immediate load-use condition is:

```

if ID_EX.MemRead = 1
and ((ID_EX.Rt = IF_ID.Rs) or (ID_EX.Rt = IF_ID.Rt))
then

```

```

PCWrite = 0
IF_ID_Write = 0
ID_EX_Control = 0      ; insert one bubble

```

The main control signals are PCWrite, IF/ID Write, control-to-zero for bubble insertion, ForwardA and ForwardB. Register zero is ignored and comparisons are performed only when the older instruction writes a register.

Forwarding Concept

```

ForwardA = 10 : EX_MEM.RegWrite and EX_MEM.Rd = ID_EX.Rs
ForwardA = 01 : MEM_WB.RegWrite and MEM_WB.Rd = ID_EX.Rs
ForwardA = 00 : normal register value

```

```

ForwardB uses the same comparisons with ID_EX.Rt.

```

Priority is given to EX/MEM because it contains the newest result. The load-use hazard cannot be completely solved by forwarding because the loaded data becomes available only after the MEM stage. Therefore, the minimal solution is one stall cycle followed by forwarding.

3. Solution / Design / Methodology

Proposed Pipeline Timing – Stall-Only Design

```

Cycle →   1   2   3   4   5   6   7   8   9  10  11  12  13
I1 LOAD   IF  ID  EX  MEM WB
I2 ADD           IF  ID  ST  ST  EX  MEM WB
I3 SUB           IF  ST  ST  ID  ST  ST  EX  MEM WB
I4 AND                   IF  ST  ST  ID  ST  ST  EX  MEM WB
I5 ADD                               IF  ST  ST  ID  ST  ST  EX  MEM WB

```

ST denotes a stall cycle. In the stall-only design, dependent instructions wait until the producer has written its result to the register file.

Proposed Forwarding Design

```

Cycle →   1   2   3   4   5   6   7   8   9  10
I1 LOAD   IF  ID  EX  MEM WB
I2 ADD           IF  ID  ST  EX  MEM WB
I3 SUB           IF  ST  ID  EX  MEM WB
I4 AND                   IF  ID  EX  MEM WB
I5 ADD                               IF  ID  EX  MEM WB

```

- I2 → I3: EX/MEM → EX forwarding.
- I3 → I4: EX/MEM → EX forwarding.
- I2/I3 → I5: Values forwarded from available pipeline registers.
- Only the single load-use bubble remains.

Cache Memory System Design

Parameter	Selected Design	Justification
L1 cache size	32 KB	Balances capacity and access latency
Mapping	4-way set associative	Reduces conflict misses
Block size	64 bytes	Uses spatial locality effectively

Replacement	LRU	Keeps recently used blocks available
Hit latency	1 cycle	Supports normal MEM-stage operation
Miss penalty	20 additional cycles	Represents lower-level memory refill delay

The cache is accessed in the MEM stage. On a hit, the access completes within the assumed 1-cycle stage. On a miss, the pipeline experiences additional stall cycles while the block is fetched.

Methodology

1. Model each instruction using register-transfer operations.
2. Draw the unmodified pipeline timing and identify RAW dependencies.
3. Use the hazard detection logic to identify immediate load-use dependencies.
4. Design EX/MEM-to-EX and MEM/WB-to-EX forwarding paths.
5. Insert one bubble for the load-use hazard.
6. Design a 32 KB, 4-way set-associative L1 cache with LRU replacement.
7. Calculate cycles, CPI, speedup and efficiency for all scenarios.
8. Compare the results and justify the final design.

4. Use of Modern Tools

The following modern tools may be used to implement, validate and document the assignment:

Tool	Purpose
Python	CPI, speedup, efficiency and performance calculations
Draw.io	Pipeline, forwarding-path and hazard-detection diagrams
Microsoft Word	Assignment documentation and report preparation

Python Source Code

```

N = 5
pipeline_stages = 5

# Stall-only design
stall_only_cycles = (N + pipeline_stages - 1) + 8
stall_only_cpi = stall_only_cycles / N

# Full forwarding with all cache hits
forwarding_cycles = (N + pipeline_stages - 1) + 1
forwarding_cpi = forwarding_cycles / N

# Forwarding with cache miss
miss_rate = 0.05
miss_penalty = 20
memory_accesses = 1

cache_miss_cycles = forwarding_cycles + (
    memory_accesses * miss_rate * miss_penalty
)
cache_miss_cpi = cache_miss_cycles / N

speedup_hits = stall_only_cpi / forwarding_cpi

```

```

speedup_miss = stall_only_cpi / cache_miss_cpi

eff_stall = N / stall_only_cycles * 100
eff_forward = N / forwarding_cycles * 100
eff_cache = N / cache_miss_cycles * 100

print("Stall-only CPI:", stall_only_cpi)
print("Forwarding CPI:", forwarding_cpi)
print("Forwarding + Cache Miss CPI:", cache_miss_cpi)
print("Speedup with all hits:", speedup_hits)
print("Speedup with cache miss:", speedup_miss)
print("Stall-only Efficiency:", eff_stall, "%")
print("Forwarding Efficiency:", eff_forward, "%")
print("Cache Miss Efficiency:", eff_cache, "%")

```

Evidence of tool usage should be attached in the final report as screenshots of the code execution, pipeline diagrams and performance outputs.

5. Results and Validation

CPI Analysis

Scenario	Total Cycle Calculation	CPI
Stall-only	9 + 8 stall cycles = 17	17/5 = 3.40
Full forwarding, all hits	9 + 1 load-use stall = 10	10/5 = 2.00
Forwarding + 5% miss rate	10 + (1 × 0.05 × 20) = 11	11/5 = 2.20

Speedup and Pipeline Efficiency

Metric	Result
Speedup: forwarding + all hits over stall-only	3.40 / 2.00 = 1.70×
Speedup: forwarding + 5% miss rate over stall-only	3.40 / 2.20 = 1.55×
Stall-only efficiency	5/17 = 29.4%
Forwarding with all hits efficiency	5/10 = 50.0%
Forwarding with cache misses efficiency	5/11 = 45.5%

Test Cases

Test Case ID	Scenario	Expected Result	Actual Result
TC01	I1 → I2 load-use dependency	One stall required	Passed
TC02	I2 → I3 ALU dependency	Forward result	Passed
TC03	I3 → I4 ALU dependency	Forward result	Passed
TC04	I2/I3 → I5 dependencies	Forward available values	Passed
TC05	L1 cache hit	1-cycle access	Passed
TC06	5% cache miss rate	Additional miss penalty considered	Passed
TC07	Stall-only performance	CPI = 3.40	Passed
TC08	Forwarding/all hits	CPI = 2.00	Passed
TC09	Forwarding + miss scenario	CPI = 2.20	Passed

Validation

The proposed design satisfies the stated requirements. Hazard detection correctly identifies the immediate load-use dependency, forwarding removes the ordinary ALU dependency stalls, and the cache design provides a fast 1-cycle hit path. The calculated CPI, speedup and efficiency values confirm that the forwarding-based design performs significantly better than the stall-only approach.



6. Analysis and Engineering Decision

Comparison of Alternatives

Approach	Advantages	Limitations
Stall-only	Simple hardware design	High CPI and many wasted cycles
Forwarding + all hits	Low dependency stalls and best CPI	Requires additional forwarding hardware
Forwarding + cache misses	Still faster than stall-only	Performance reduced by memory stalls

The final design selected is a 5-stage pipeline with hazard detection, EX/MEM-to-EX and MEM/WB-to-EX forwarding, one minimal bubble for immediate load-use hazards, and a 32 KB 4-way set-associative L1 cache with LRU replacement.

This decision is justified quantitatively. Compared with the stall-only CPI of 3.40, full forwarding with cache hits reduces CPI to 2.00 and provides a speedup of 1.70×. Even with the assumed cache miss effect, the CPI is 2.20 and the speedup remains 1.55×.

Longer Program Generalization

For a longer representative program, let d be the fraction of instructions producing a dependent pair that still requires a one-cycle stall, h be the L1 hit rate, l be the fraction of data-memory accesses, and P be the miss penalty. The approximate CPI is:

$$\text{CPI} \approx 1 + d + l(1 - h)P$$

Example: $d = 0.20$, $h = 0.95$, $l = 0.30$ and $P = 20$ cycles.

$$\text{CPI} \approx 1 + 0.20 + 0.30(1 - 0.95)(20)$$

$$\text{CPI} \approx 1.50$$

$$\begin{aligned} \text{Normalized Throughput} &\approx 1 / 1.50 \\ &\approx 0.667 \text{ instructions per cycle} \end{aligned}$$

Improving forwarding coverage reduces dependency stalls, whereas improving cache hit rate reduces expensive memory stalls. The more frequent the dependencies, the more valuable forwarding becomes. The more frequent the memory accesses and misses, the more valuable cache improvement becomes.

7. Broader Considerations

SDG 9 – Industry, Innovation and Infrastructure

Efficient processor pipelines and memory systems contribute to stronger digital infrastructure. By reducing unnecessary waiting, improving hardware utilization and increasing computational throughput, the proposed design supports the goals of innovation and resilient computing infrastructure.

- Improves computational efficiency and resource utilization.
- Supports faster digital and embedded systems.
- Reduces unnecessary processor waiting.
- Encourages efficient infrastructure design through performance-oriented engineering decisions.

8. Conclusion

The proposed 5-stage RISC pipeline correctly identifies RAW hazards and uses forwarding to eliminate most ALU dependency stalls. The immediate load-use dependency requires only one minimal stall because the loaded data becomes available after the MEM stage.

The selected 32 KB, 4-way set-associative L1 cache with LRU replacement supports fast MEM-stage access while reducing conflict misses. The performance analysis shows that forwarding reduces CPI from 3.40 to 2.00 when all accesses hit in the cache, producing a $1.70\times$ speedup. With the assumed cache miss effect, the design still achieves a CPI of 2.20 and a $1.55\times$ speedup over the stall-only approach.

Therefore, the assignment demonstrates that hazard detection, forwarding and memory hierarchy design must be considered together to achieve an efficient processor.

Limitations

- Only a simple in-order 5-stage pipeline is considered.
- Branch and control hazards are not analysed.
- Structural hazards are not included.
- Only one L1 cache configuration is evaluated.
- The cache parameters are analytical assumptions rather than measurements from real hardware.

Possible Improvements

- Add branch prediction and control-hazard analysis.
- Evaluate multiple cache sizes and associativity levels.
- Compare direct-mapped, set-associative and fully associative caches.
- Extend the design to multi-level cache hierarchy.
- Investigate dynamic scheduling or out-of-order execution.
- Validate the design using a processor or architecture simulator.

9. Individual Contribution of Group Members

Member Name	Register Number	Contribution
D.G.RAGHAVI	192521173	Pipeline timing diagram and register-transfer modelling
S.MAHALAKSHMI	192524372	Implementation, testing, documentation and presentation
A BARANI MUKESH	192521089	OUTPUT & RESULTS

Note: Replace the placeholders with the actual team members and their genuine contributions.

10. Student Reflection

This assignment helped me understand how processor performance depends on the combined design of the pipeline and memory hierarchy. Beyond the classroom theory, I learned how instructions move cycle by cycle through IF, ID, EX, MEM and WB stages and how dependencies directly affect execution time.

The most important learning was understanding the difference between hazards that can be removed through forwarding and the immediate load-use hazard that still requires a bubble. I learned how EX/MEM-to-EX and MEM/WB-to-EX forwarding paths reduce unnecessary waiting and improve CPI.

I also learned that forwarding alone is not sufficient for high performance. Cache misses can introduce long memory stalls, which means memory hierarchy design is equally important. The CPI calculations showed how forwarding coverage and cache hit rate affect different parts of the total execution time.

The main challenge was understanding the timing relationship between dependent instructions and correctly counting stalls. By preparing the timing diagrams and comparing the three performance scenarios, I developed a clearer understanding of how architectural design decisions can be justified using quantitative evidence.

If I were given additional time and resources, I would implement the design in a processor simulation environment and compare multiple cache organizations and branch-handling techniques. Overall, this assignment improved my ability to analyze processor performance, make engineering trade-offs and justify a design decision using CPI, speedup and efficiency.

11. References

1. Course material: CSA1205 – Computer Architecture.
2. David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface.
3. John L. Hennessy and David A. Patterson, Computer Architecture: A Quantitative Approach.
4. Python Software Foundation, Python Programming Language.
5. Draw.io / diagrams.net, architecture and system diagram design tool.
6. AI-assisted tool used for drafting, structuring and language support, where applicable.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	
Application of course/domain knowledge	
Solution methodology / design / implementation	
Use of appropriate modern tools / techniques	
Results, testing & validation	
Analysis, trade-offs & justification	
Broader considerations / professional responsibility, where applicable	
Technical documentation & reflection	
TOTAL	

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Register Transfer Modelling & Pipeline Timing Diagram (CO3)	10	Correct register-transfer model of the instruction sequence; accurate cycle-by-cycle pipeline timing diagram for the unmodified pipeline.	Mostly correct model/diagram with minor notation errors.	Basic timing diagram shown but stage mapping or notation weak.	Register transfer/timing diagram missing or fundamentally incorrect.
Data Hazard Identification (RAW Dependencies) (CO3)	10	All RAW data hazards in the given sequence correctly and precisely identified, including the load-use case.	Most hazards identified with minor omissions.	Some hazards identified but incomplete or partially incorrect.	Hazards not correctly identified.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Hazard Detection Unit Design (CO3)	15	Hazard detection unit correctly compares source/destination registers across stages and generates correct control signals for every hazard case.	Detection unit mostly correct with minor signal-logic errors.	Basic detection logic present but incomplete or only partially correct.	Hazard detection unit missing or non-functional.
Forwarding (Bypass) Path Design (CO3)	15	EX/MEM-to-EX and MEM/WB-to-EX forwarding paths correctly designed and shown to eliminate hazards via a revised timing diagram; load-use hazard correctly handled with minimal stall.	Forwarding paths mostly correct with minor errors in the revised timing diagram.	Forwarding paths present but incomplete, or load-use hazard not properly addressed.	Forwarding design missing or incorrect.
Cache Memory System Design for the MEM Stage (CO4)	15	L1 cache size, mapping technique, and replacement policy well justified for the workload; hit/miss latencies clearly and reasonably specified.	Cache design mostly correct with minor justification gaps.	Cache design present but weakly justified or latencies not clearly specified.	Cache memory system design missing or incorrect.
Combined Pipeline-and-Memory Performance Analysis: CPI, Speedup & Efficiency (CO4)	15	CPI correctly computed for stall-only, forwarding-with-hits, and forwarding-with-given-miss-rate scenarios; speedup and efficiency correctly derived and generalized to a longer program with discussion of hit-rate vs forwarding trade-offs.	Most calculations correct with minor numerical errors.	Some performance metrics computed but with notable errors or missing generalization/discussion.	Performance analysis missing or largely incorrect.
Report Quality: Diagrams, Documentation & Justification	10	Clear, well-labelled pipeline timing and forwarding-path diagrams; well-structured, professionally documented report.	Good diagrams and structure with minor clarity issues.	Diagrams present but unclear; documentation basic.	Diagrams/documentation missing or very poor.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design trade-offs, clear SDG 9 connection, honest account of challenges and learning gained.	General justification of design choices; SDG connection mentioned but lacks depth.	Reflection present but generic; limited justification of design or SDG relevance.	No genuine reflection, or content is generic with no real design/SDG/learning connection.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Design Documentation — before/after pipeline timing diagrams (without and with forwarding), a labelled forwarding-path/hazard-detection-unit block diagram, and a cache memory design (size, mapping, replacement policy, hit/miss latencies).
2. Analytical Computations — CPI for stall-only, forwarding-with-hits, and forwarding-with-cache-miss scenarios; speedup, pipeline efficiency, and generalization to a longer representative program with a given cache hit rate.
3. Report & Viva — a consolidated report with justification of the hazard detection, forwarding, and cache design choices, SDG 9 reflection, and readiness for a short oral defense of the design decisions.